



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Ángel Julián García López **Carné:** 1097925

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
R/ Es una interfaz que permite que diferentes aplicaciones se comuniquen mediante HTTP.
2. ¿Qué es un endpoint?
R/ Es una URL específica donde una API recibe o envía información.
3. ¿Qué diferencia existe entre una URL y un endpoint?
R/ Una URL identifica un recurso en Internet; un endpoint es una URL específica de una API para realizar una función.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
R/ GET: obtener datos.
POST: crear datos.
PUT/PATCH: actualizar datos.
DELETE: eliminar datos.
5. ¿Qué es JSON y por qué es común en las APIs?
R/ Es un formato para organizar datos. Es común porque es sencillo, ligero y fácil de leer.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
R/ Indica el resultado de una solicitud. Ejemplos: 200 = correcto, 404 = no encontrado, 500 = error del servidor.
7. ¿Qué información se puede encontrar en la respuesta de una API?
R/ Datos solicitados, código de estado, mensajes y, en ocasiones, información sobre errores.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
R/ Permite probar solicitudes, revisar respuestas y detectar errores antes de programar.
9. ¿Qué es y para qué sirve Swagger UI?
R/ Es una herramienta que permite visualizar y probar la documentación de una API.
10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?
R/ Sirve para desarrollar, probar y verificar el funcionamiento de APIs.



1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
`https://petstore3.swagger.io/api/v3/{petId}`
- Indique el método HTTP utilizado.
GET
- Indique qué parámetro necesita la solicitud.
petId
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
`https://petstore3.swagger.io/api/v3/pet/1`
- Explique con sus palabras qué información contiene la respuesta.
Debería de contener la información del pet con id numero "1"

Evidencia: captura de Swagger mostrando la operación y su respuesta.

The screenshot shows the Swagger UI interface. At the top, the endpoint `petId` is defined as a required integer (path). Below this, the `Execute` button is highlighted. The `Responses` section shows the result of the GET request. The `Request URL` is `https://petstore3.swagger.io/api/v3/pet/1`. The `Server response` is a 200 status code. The `Response body` is a JSON object:

```
{  "id": 1,  "name": "Pet1",  "photoUrls": [    "test1",    "test2"  ],  "tags": [],  "status": "available"}
```

. The `Response headers` include `access-control-allow-headers`, `access-control-allow-methods`, `access-control-allow-origin`, `access-control-expose-headers`, `content-length`, `content-type`, `date`, and `server`.



3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

Request body required

application/json

Create a new pet in the store

Edit Value | Schema

```
{
  "id": 67,
  "name": "cobito",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Server response

Code	Details	
200	Response body <pre>{ "id": 67, "category": { "id": 1, "name": "Dogs" }, "name": "cobito", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "available" }</pre> Response headers<pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 137 content-type: application/json date: Sat,03 Oct 2026 02:45:43 GMT server: Jetty(9.4.53.v20231009)</pre>	
Responses		
Code	Description	Links
200	Successful operation	No links



petId * required ID of pet to return
integer(\$int64)
(path)

67

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/67' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/67

Server response

Code	Details
200	Response body <pre>{ "id": 67, "category": { "id": 1, "name": "Dogs" }, "name": "cobito", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "available" }</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 137 content-type: application/json date: Sat, 03 Oct 2026 02:46:31 GMT server: Jetty(9.4.53.v20231009)</pre>

Responses

Code	Description	Links
------	-------------	-------

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
R/ Utilicé el método PUT, porque permite actualizar un recurso existente.
- ¿Qué datos fueron enviados en el body?
R/ Se envió la mascota con el ID 10, cambiando el nombre “Doggie” a “Perrito” y el estado “Available” a “Disponible”
- ¿Qué código HTTP devolvió la API?
Devolvió el 200 OK porque se realizó correctamente
- ¿Cómo comprobó que el cambio realmente ocurrió?
Volví a hacer un GET y me di cuenta que si se cambio la información



https://petstore3.swagger.io/api/v3/pet

Server response

Code	Details
200	Response body <pre>{ "id": 10, "category": { "id": 1, "name": "Dogs" }, "name": "Perrito", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "disponible" }</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 139 content-type: application/json date: Sat,03 Oct 2026 02:48:46 GMT server: Jetty(9.4.53.v20231009)</pre>

Responses

Code	Description	Links
200	Successful operation	No links

Media type
application/json

Controls Accept header:

Example Value | Schema

```
{
  "id": 10,
  "name": "doggie",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

400 Invalid ID supplied

No links

Curl

```
curl -X 'GET' \
'https://petstore3.swagger.io/api/v3/pet/10' \
-H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/10

Server response

Code	Details
200	Response body <pre>{ "id": 10, "category": { "id": 1, "name": "Dogs" }, "name": "Perrito", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "Disponible" }</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 139 content-type: application/json date: Sat,03 Oct 2026 02:54:52 GMT server: Jetty(9.4.53.v20231009)</pre>

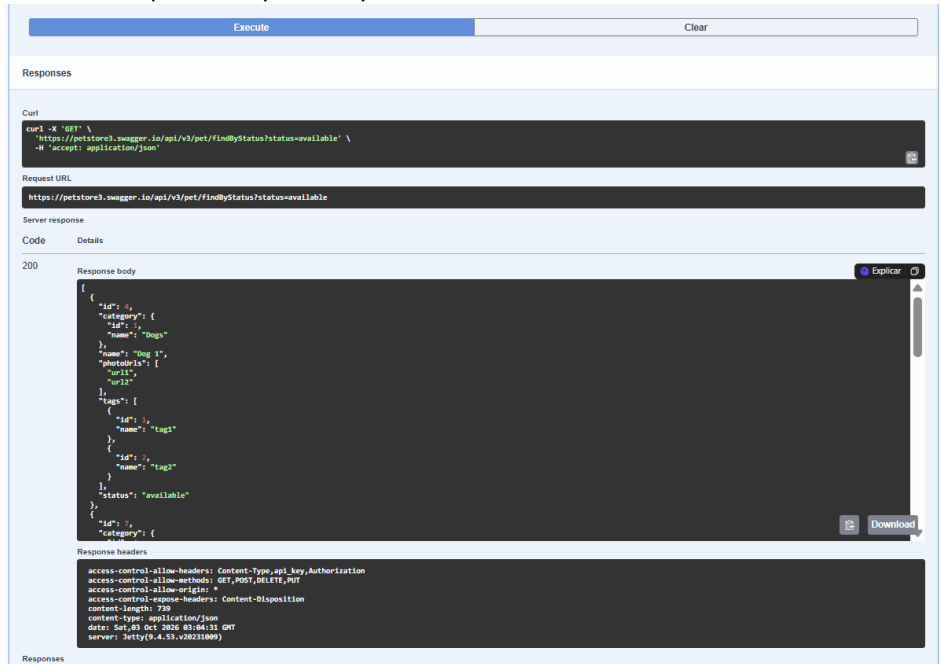
Responses

Code	Description	Links
200	Successful operation	No links



5. Consultar utilizando parámetros

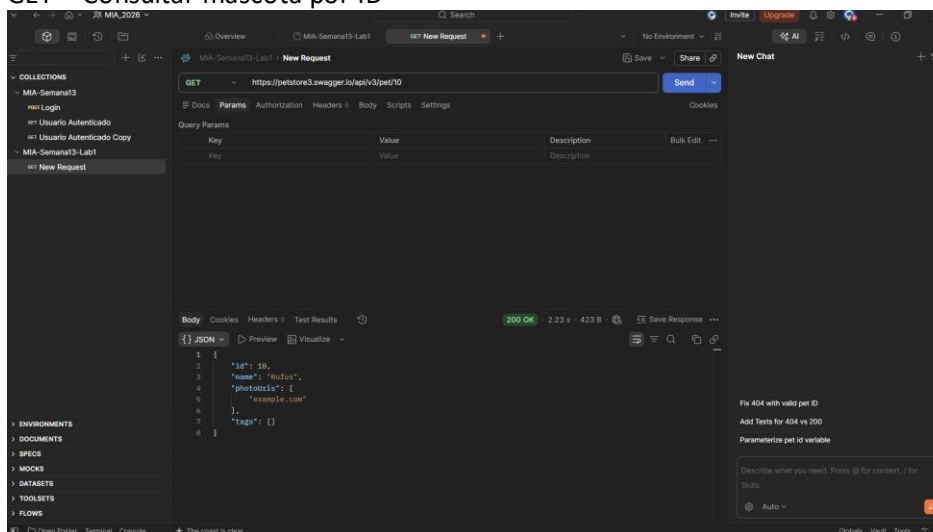
- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.



- Identifique el parámetro utilizado.
El parámetro utilizado fue `findByStatus?status=available`
- Explique dónde se envía ese parámetro.
Se envía en el endpoint después del path
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por `pending`.
Cambia los valores a las mascotas que tienen status "pending"

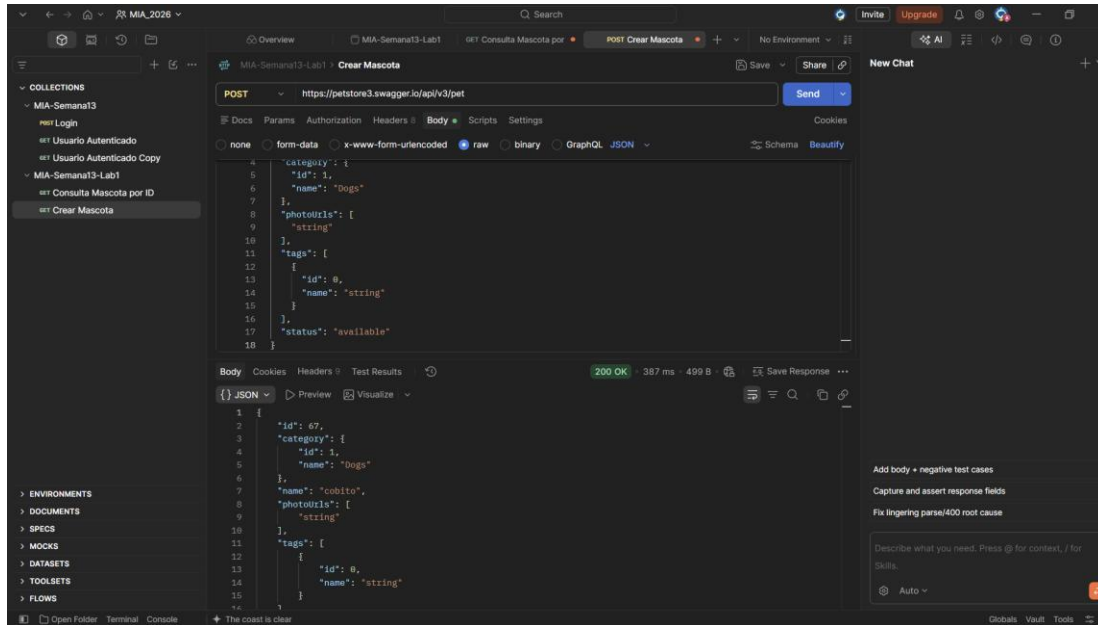
6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID

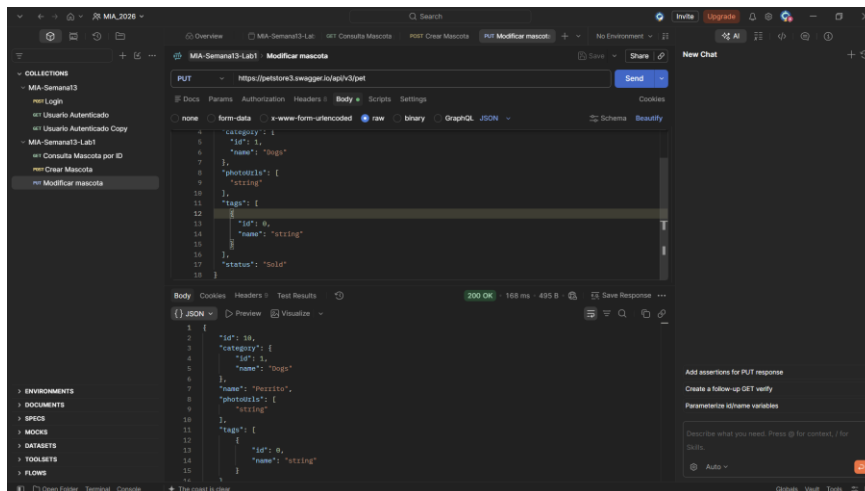




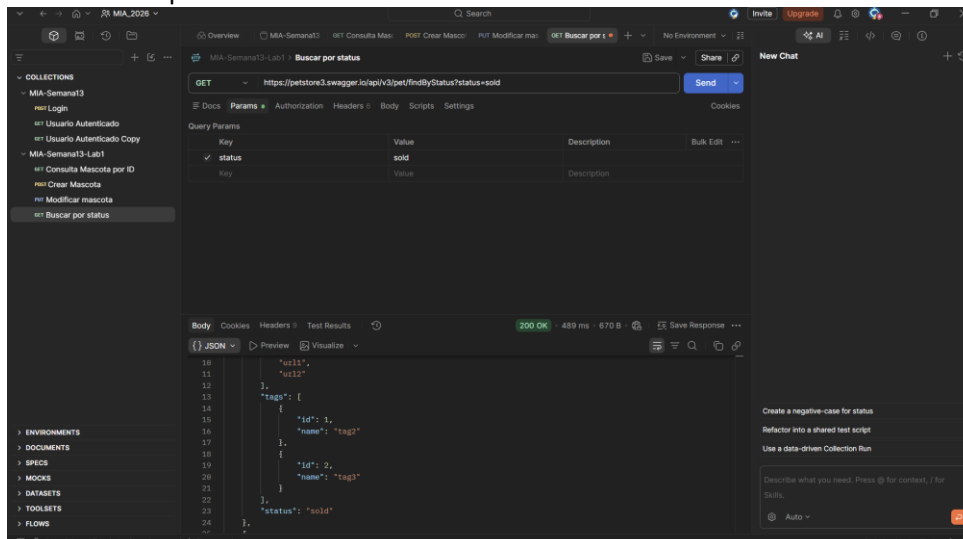
- POST – Crear mascota



- PUT – Modificar mascota

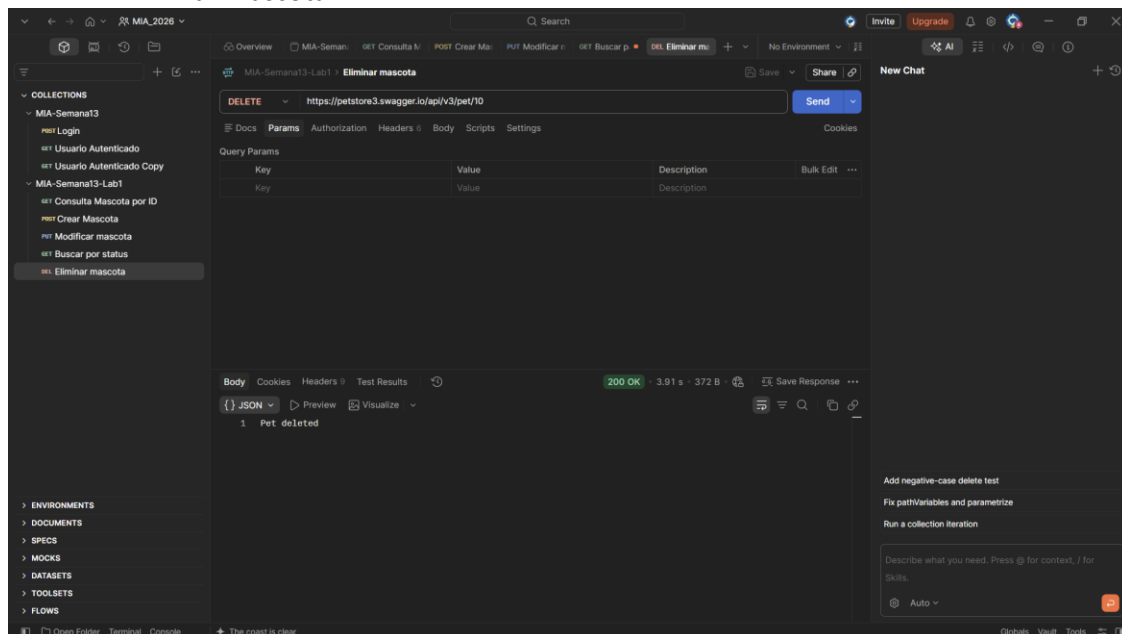


- GET – Buscar por status





- DELETE – Eliminar mascota



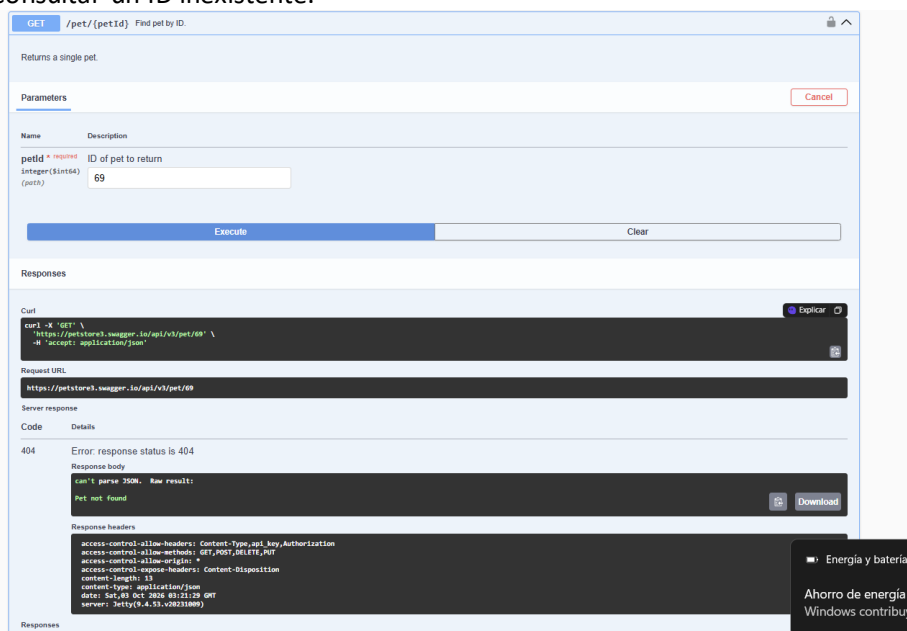
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

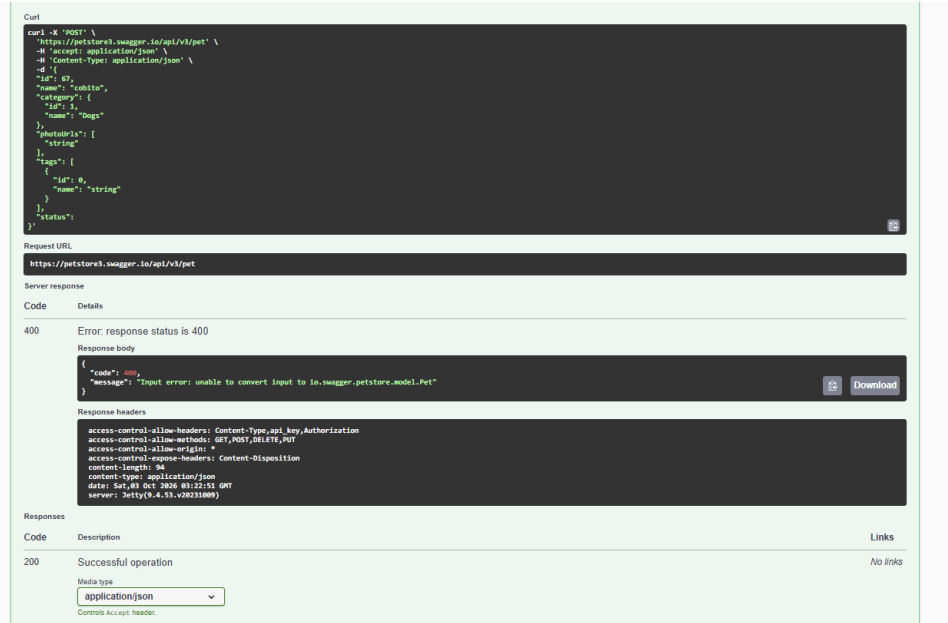
Ejemplos de escenarios:

- Consultar un ID inexistente.



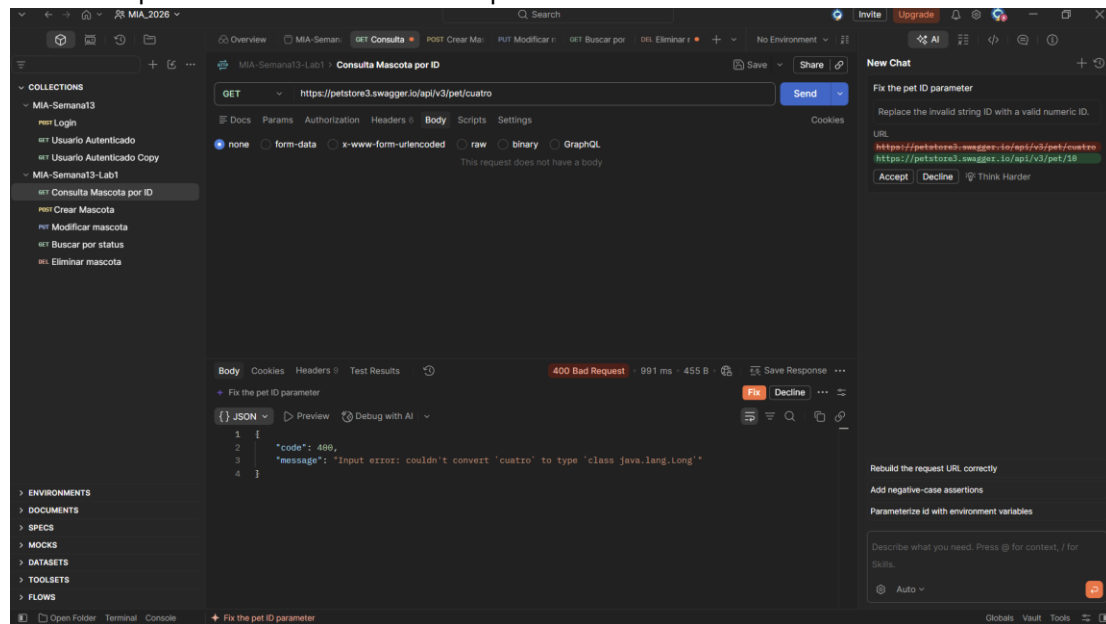
Como se sabe el error 404 es not found, como bien esperábamos no se encontró ninguna mascota que tuviera id 69

2. Enviar un JSON incorrecto o incompleto.



La API devolvió un error 400 porque los datos enviados en el body no tenían un formato válido para crear la mascota.

3. Utilizar un parámetro con un valor no esperado.



No introducimos el parámetro en un formato valido por eso nos da error

4. Intentar eliminar un recurso que ya no existe.



The screenshot shows a REST client interface with the following sections:

- Parameters:** A table with columns 'Name' and 'Description'. It contains two entries: 'api_key' (string, header) with a value 'api_key', and 'petid' (integer, path) with a value '10'. The 'petid' entry is marked as required.
- Buttons:** 'Execute' (blue) and 'Clear' (grey).
- Responses:** A section showing the results of the request.
 - Curl:** A code block containing the command: `curl -X 'DELETE' \n https://petstore3.swagger.io/api/v3/pet/10' \n -H 'accept: */*'`
 - Request URL:** A text field showing `https://petstore3.swagger.io/api/v3/pet/10`
 - Server response:** A section showing the response details.
 - Code:** 200
 - Response body:** A code block showing `can't parse JSON. Raw result: \n Pet deleted`
 - Response headers:** A code block showing various headers including `access-control-allow-headers`, `access-control-allow-methods`, `access-control-allow-origin`, `access-control-expose-headers`, `content-length`, `content-type`, `date`, and `server`.

No paso absolutamente nada, solo vuelve a mostrar http de 200 OK

Para cada prueba registre:

- Solicitud realizada.
- Código HTTP recibido.
- Respuesta de la API.
- Qué esperaba que ocurriera.
- Qué ocurrió realmente.
- Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.



Postman interface showing a POST request to `https://petstore3.swagger.io/api/v3/pet`. The request body is a JSON object:

```
{  "category": {    "id": 1,    "name": "Dogs"  },  "photoUrls": [    "string"  ],  "tags": [    {      "id": 1,      "name": "MIA"    }  ],  "status": "available"}
```

The response is `200 OK` with a status of `2.93 s` and `501 B`. The response body is a JSON object:

```
{  "id": 100,  "category": {    "id": 1,    "name": "Dogs"  },  "name": "PerritoMIA",  "photoUrls": [    "string"  ],  "tags": [    {      "id": 1,      "name": "MIA"    }  ]}
```

Postman interface showing a GET request to `https://petstore3.swagger.io/api/v3/pet/100`. The response is `200 OK` with a status of `105 ms` and `501 B`. The response body is a JSON object:

```
{  "id": 100,  "category": {    "id": 1,    "name": "Dogs"  },  "name": "PerritoMIA",  "photoUrls": [    "string"  ],  "tags": [    {      "id": 1,      "name": "MIA"    }  ]}
```



Postman interface showing a PUT request to `https://petstore3.swagger.io/api/v3/pet/` with a JSON body. The response is `200 OK` with a status of `88 ms` and a body size of `496 B`. The JSON body contains pet information: `{ "id": 100, "name": "PerritoMIA", "category": { "id": 1, "name": "Dogs" }, "photoUrls": [ "string" ], "tags": [ { "id": 1, "name": "MIA" } ] }`.

Postman interface showing a DELETE request to `https://petstore3.swagger.io/api/v3/pet/100`. The response is `200 OK` with a status of `156 ms` and a body size of `372 B`. The JSON body contains the message: `{ "id": 1, "name": "Dogs" }, { "name": "PerritoMIA", "photoUrls": [ "string" ], "tags": [ { "id": 1, "name": "MIA" } ], "status": "sold" }`.

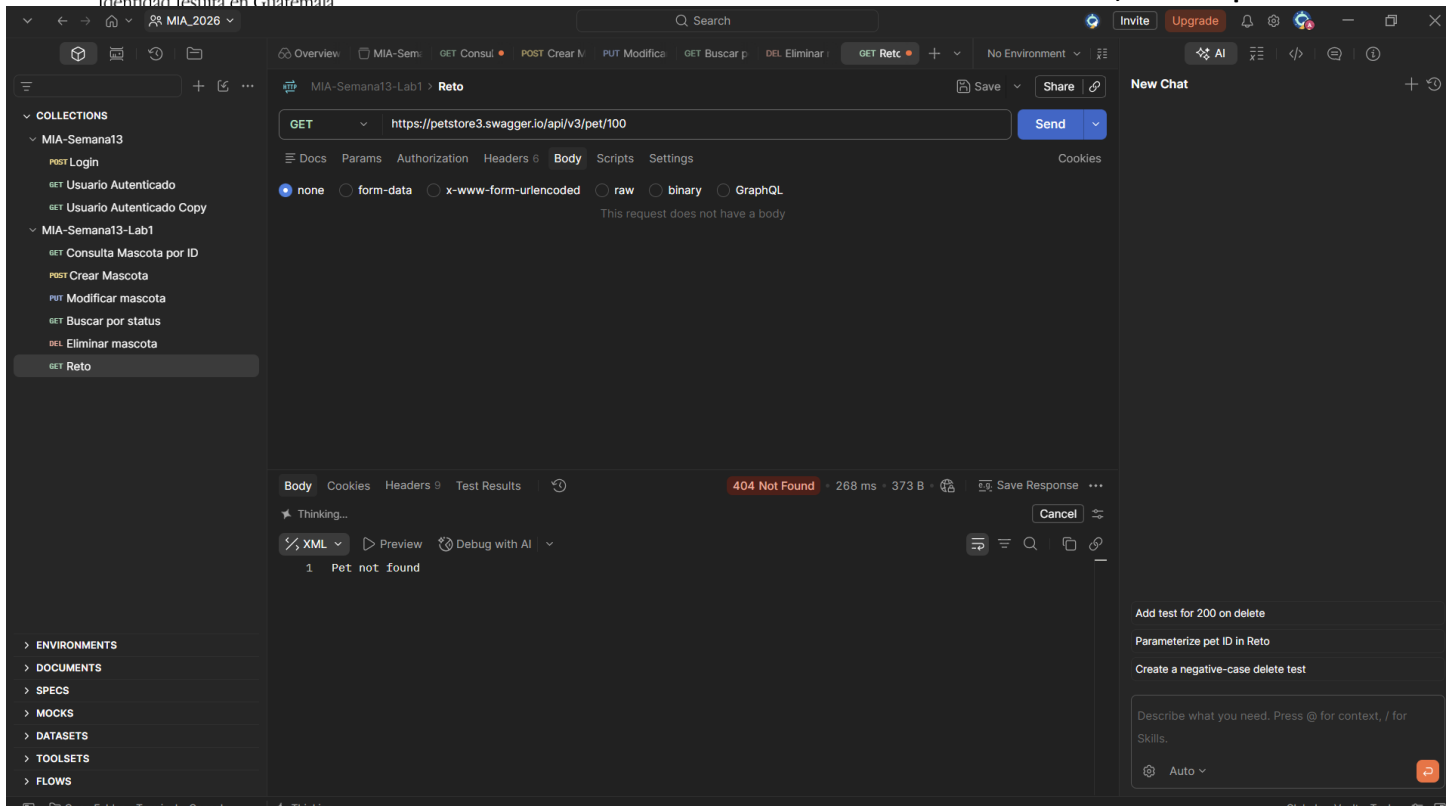


FOTO DE PRUEBA EN CLASE

